ERAS POST-OP MONITORING · RULES ENGINE

Rule configuration: where the scoring rules are stored

Ruleset TKR 1.0.0 · 19 September 2026

> The rules live in the database, not in code. The engine in `backend/app/services/rules_engine.py` only interprets them.

The tables

TABLE	WHAT IT HOLDS
`meta.ruleset`	One row per version, e.g. TKR `1.0.0`. It holds the status (`draft` / `approved` / `retired`), the tier cut-offs (`review_at` = 5, `critical_at` = 12), `max_points_per_parameter` = 6, and who approved it. A partial unique index allows at most one approved ruleset per procedure.
`meta.procedure_parameter`	One row per ruleset × measure. It holds the question wording, when the question is asked (`collect_from_day` / `collect_to_day`), when the answer counts (`score_from_day` / `score_to_day`), the `parameter_weight`, and the rule itself in `scoring_config` (JSONB).
`meta.parameter`	The measure itself: its type (numeric / boolean / ordinal), unit and valid range, and which yes/no answer is the worrying one.
`meta.parameter_option`	Ranked options for the ordinal measures (walking aid, knee bend).

The rule logic: `scoring_config`

A band is `[from, to, points]`. It matches when `from ≤ value < to`, and a value outside every band scores 0.

`temp_c` — Scored today

```
{"bands": [[0,37.5,0],[37.5,38.0,1],[38.0,38.5,2],[38.5,99,3]]}
```

Fixed bands: 0 points below 37.5 °C, rising to 3 points at 38.5 °C and above.

`pain` — Scored today

```
{"bands_by_day": {"0–3":  [[0,8,0],[8,10,1],[10,99,3]],
                  "4–10": [[0,4,0],[4,6,1],[6,8,2],[8,99,3]],
                  "11–84":[[0,3,0],[3,5,1],[5,7,2],[7,99,3]]}}
```

The bands depend on the post-op day, so pain that is expected on day 2 becomes a concern by day 12.

`hr` — Scored today

```
{"relative_to_baseline": true, "bands": [[0,10,0],[10,20,1],[20,30,2],[30,999,3]]}
```

Scored on the rise above the patient's discharge heart rate. With no discharge baseline, it scores nothing.

`breathless` Scored today

```
{"binary_score": 3, "emergency": true}
```

A “yes” is an emergency and bypasses the weighted total. A red flag at `binary_score` 3 without `emergency` (e.g. calf pain) is critical.

`steps` Not yet implemented

```
{"detector": "drop_from_personal_best", "drop_fraction": 0.5, "score": 2}
```

A trend detector: it flags when steps fall to half of the patient's personal best.

`knee_bend` Not yet implemented

```
{"milestone": {"14": "chair", "42": "car", "84": "bike"}, "score": 2}
```

A recovery milestone: the patient is expected to reach each level by the given day. `walk_aid` uses the same shape.

The engine currently acts on `bands`, `bands_by_day`, `relative_to_baseline`, `binary_score` and `emergency`. It ignores `detector` and `milestone`: those rules are configured but the trend detectors and milestones have not been built yet.

How it ties together

- **Episodes are pinned to a version.** Each patient's `txn.patient_episode.ruleset_id` fixes which version scores them, and the engine uses that rather than whatever is approved now. Approving a new version does not silently re-score patients who are already being monitored.
- **Every decision records its version.** Each `txn.check_in_evaluation` row stores the `ruleset_id` it was scored under, with its reasons, so any past alert can be explained.
- **The source of truth in the repository** is `db/seed/01_reference.sql`. Changes to a live database go through a numbered migration in `db/migrations/`. For example, `002_remove_redness_parameter.sql` removed the redness rule.
- **You can view the rules in the app** on the Rulesets screen (`/rulesets`), which is read-only by design.

Source: `db/schema/01_meta.sql`, `db/seed/01_reference.sql`, `backend/app/services/rules_engine.py`